

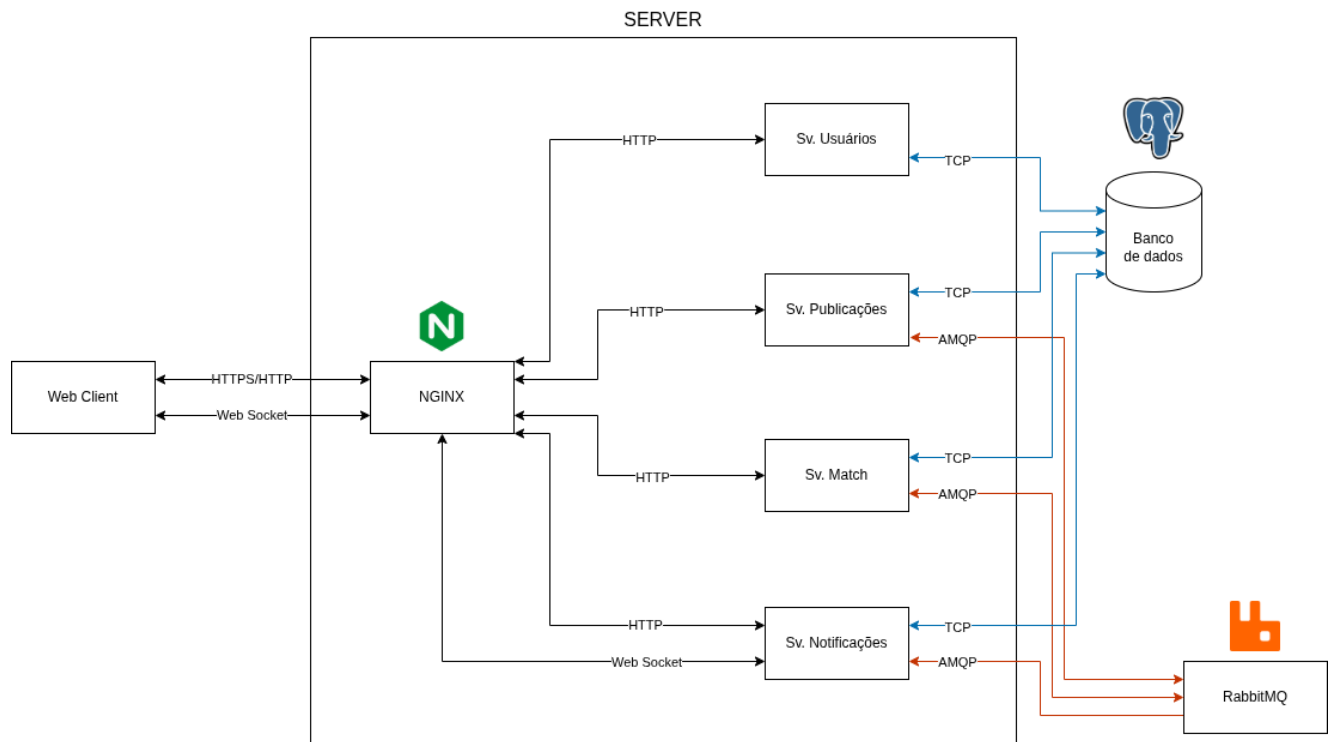
## Alunos e RA's:

Alexandre Borges Baccarini Junior - 2515520

Leonardo Naime Lima - 2515660

# Descrição arquitetural do sistema

## 1. Diagrama da arquitetura do sistema



O sistema segue uma arquitetura de microsserviços, cada serviço de backend é uma aplicação independente construída com o framework NestJS sobre Node.js, com build e execução isolados em contêineres Docker.

A comunicação síncrona entre o cliente e os serviços ocorre via HTTPS/HTTP através de um NGINX (proxy reverso). Já a comunicação assíncrona entre serviços de backend ocorre via AMQP, usando o RabbitMQ. E as notificações em tempo real ao cliente ocorrem via WebSocket, usando a biblioteca Socket.IO.

## 2. Componentes

- **Web Client:** Interface na qual o usuário interage na web, foi implementada em React. O WEB Client interage com o sistema a partir de chamadas HTTPS para o serviço do NGINX. A sessão é mantida via JWT (JSON Web Token) armazenado em local storage também mantém uma conexão WebSocket persistente com o Sv. Notificação para receber notificações em tempo real.

- **NGINX:** Atua como proxy reverso e ponto único de entrada do sistema para comunicação com o cliente, encaminhando as requisições para o serviço apropriado.
- **Banco de Dados (PostgreSQL):** Sistema gerenciador de banco de dados relacional acessado pelos quatro microsserviços via protocolo TCP.
- **RabbitMQ:** Broker de mensagens responsável pela comunicação assíncrona entre o Sv. Publicações, o Sv. Match e o Sv. Notificação, via protocolo AMQP. Os três serviços trocam eventos por meio de um único exchange do tipo topic, permitindo múltiplos consumidores para o mesmo evento. Os eventos trocados são:
  - Publicados pelo Sv. Publicações e consumidos pelo Sv. Match: ``publicacao.criada``, ``publicacao.atualizada``, ``publicacao.removida``.
  - Publicados pelo Sv. Match e consumidos pelo Sv. Publicações e pelo Sv. Notificação: ``match.encontrado``, ``match.aceito``, ``match.recusado``, ``match.expirado``, ``match.cancelado``.
- **Sv. Usuários:** Serviço responsável pelo cadastro, autenticação e gerenciamento de perfil dos usuários, implementado utilizando o NestJS. Recebe as requisições pelo NGINX e se comunica com a Base de Dados para persistir e consultar os dados dos usuários. É o serviço responsável por emitir o token JWT utilizado nas demais chamadas autenticadas do sistema.
- **Sv. Publicações:** Serviço responsável pelo cadastro e gerenciamento das publicações de itens oferecidos para troca. Recebe as requisições do NGINX e persiste os dados na Base de Dados. Ao criar, atualizar ou remover uma publicação, envia uma mensagem ao RabbitMQ sinalizando o evento, também consome os eventos de match para atualizar o estado das publicações envolvidas em uma proposta.
- **Sv. Match:** Serviço responsável pela regra de negócio de "matching" entre publicações compatíveis. Consome os eventos de publicação do RabbitMQ e busca uma publicação de outro usuário cujos itens sejam o inverso exato da publicação recebida, ao encontrar um par, cria uma proposta de troca e publica o evento correspondente no RabbitMQ. Também expõe endpoints para os usuários aceitarem ou recusarem uma proposta, e controla, junto ao RabbitMQ, a expiração de propostas não respondidas dentro do prazo.
- **Sv. Notificações:** Serviço responsável por gerar e entregar as notificações aos usuários a partir dos eventos de match. Consome os eventos do RabbitMQ, persiste as notificações no banco de dados e as entrega em tempo real aos usuários conectados por meio de uma conexão WebSocket.

### 3. Fluxos

- **Criação de uma Publicação:** Um usuário autenticado informa o item que oferece, o item que deseja em troca, a categoria e, opcionalmente, uma descrição. O NGINX encaminha a requisição ao Sv. Publicações, que

persiste a publicação com status `disponivel` e emite o evento `publicacao.criada` no RabbitMQ.

- **Formação de um Match:** Ao consumir os eventos `publicacao.criada` e `publicacao.atualizada`, o Sv. Match busca no banco de dados uma publicação de outro usuário cujos itens sejam o inverso exato da publicação recebida. Ao encontrar um par, cria uma proposta de troca, emite o evento `match.encontrado` e agenda a expiração da proposta, caso ela não seja respondida dentro do prazo. O Sv. Publicações, ao consumir esse evento, altera o status das duas publicações envolvidas para `negociando`. O Sv. Notificação cria uma notificação para os dois usuários e a envia em tempo real via WebSocket.
- **Atualização de uma Publicação:** O dono de uma publicação `disponivel` edita seus dados. O Sv. Publicações valida que o solicitante é o dono e que a publicação ainda está disponível, aplica as alterações e emite o evento `publicacao.atualizada`, que pode iniciar uma nova busca de par pelo Sv. Match, seguindo o mesmo fluxo de formação de match.
- **Resposta a uma Proposta:** Cada um dos dois usuários envolvidos em uma proposta pendente responde aceitando ou recusando a troca. Se a resposta for de recusa, a proposta é encerrada e o evento `match.recusado` é emitido imediatamente. Se ambos os participantes aceitarem, a proposta é confirmada e o evento `match.aceito` é emitido. Caso apenas um tenha aceitado até o momento, a resposta é apenas registrada, aguardando o segundo participante.
- **Confirmação da Troca:** Ao consumir o evento `match.aceito`, o Sv. Publicações altera o status das duas publicações envolvidas para `trocado`. O Sv. Notificação cria e envia, via WebSocket, uma notificação de confirmação para os dois usuários envolvidos.
- **Recusa, Expiração ou Cancelamento de uma Proposta:** Os eventos `match.recusado`, `match.expirado` e `match.cancelado` são tratados de forma equivalente pelo Sv. Publicações, que reverte o status das publicações envolvidas de volta para `disponivel`, tornando-as elegíveis para um novo pareamento. O `match.expirado` é emitido pelo Sv. Match quando o prazo da proposta se esgota sem que ambos os participantes tenham respondido. O `match.cancelado` é emitido quando uma das publicações associadas a uma proposta pendente é removida pelo seu dono. Em todos os três casos, o Sv. Notificação envia, via WebSocket, uma notificação informando o desfecho aos dois usuários envolvidos.
- **Remoção de uma Publicação:** O dono remove uma publicação, o Sv. Publicações valida a propriedade, altera o status para `removido` e emite o evento `publicacao.removida`. Caso a publicação estivesse associada a uma proposta pendente, o Sv. Match a cancela conforme descrito no fluxo anterior.
- **Notificações:** Ao autenticar-se, o WEB Client abre uma conexão WebSocket com o Sv. Notificação, enviando o token JWT da conexão. O serviço valida o token, associa a conexão ao usuário e envia imediatamente as notificações

ainda não lidas. A partir daí, qualquer notificação criada para esse usuário é entregue em tempo real através da mesma conexão.

- **Consulta e Leitura de Notificações:** Iniciado no WEB Client, o usuário consulta suas notificações ou marca uma notificação específica como lida. O Sv. Notificação retorna as notificações do usuário autenticado ou atualiza o campo de leitura, validando que a notificação pertence ao solicitante.

## 4. Interfaces de serviço

Todos os endpoints a seguir são acessados pelo cliente através do prefixo `/api`, resolvido pelo NGINX. As respostas contêm o código de status HTTP correspondente e os dados solicitados ou uma mensagem descritiva. Os endpoints marcados como protegido exigem o cabeçalho `Authorization: Bearer <token>` com um JWT válido.

- **NGINX**
  - Interface HTTP/WebSocket
    - Parâmetros de entrada: requisição HTTP ou de WebSocket.
    - Saída: resposta HTTP ou conexão WebSocket estabelecida, repassada ao serviço de destino.
    - Mensagens de erro: não há tratamento de erro customizado; falhas de conexão retornam os códigos padrão de proxy reverso (por exemplo, 502 Bad Gateway).
- **Sv. Usuários**
  - Cadastro de usuário — POST /users/register
    - Parâmetros de entrada:
      - name: string. Nome do usuário;
      - email: string. Correio eletrônico do usuário;
      - password: string. Senha do usuário.
    - Resposta: HTTP 201, com os dados do usuário criado (sem a senha).
    - Mensagens de erro: HTTP 409, e-mail já cadastrado.
  - Login — POST /users/login
    - Parâmetros de entrada:
      - email: string. Correio eletrônico do usuário;
      - password: string. Senha do usuário.
    - Resposta: HTTP 200, com o token JWT e os dados do usuário.
    - Mensagens de erro: HTTP 401, credenciais inválidas.
  - Perfil do usuário autenticado — GET /users/me, protegido
    - Parâmetros de entrada: nenhum (identidade obtida do token).
    - Resposta: HTTP 200, com os dados completos do usuário autenticado.
    - Mensagens de erro: HTTP 401, token ausente ou inválido.
  - Perfil público de um usuário — GET /users/:id

- Parâmetros de entrada: id do usuário, na URL.
- Resposta: HTTP 200, com identificador e nome do usuário.
- Mensagens de erro: HTTP 404, usuário não encontrado.
- Atualização de perfil — PATCH /users/me, protegido
  - Parâmetros de entrada (opcionais): name; password.
  - Resposta: HTTP 200, com os dados atualizados do usuário.
  - Mensagens de erro: HTTP 401, token ausente ou inválido.

## ● Sv. Publicações

- Criação de publicação — POST /publicacoes, protegido
  - Parâmetros de entrada:
    - item\_oferto: string. Item oferecido;
    - item\_desejado: string. Item desejado em troca;
    - categoria: string;
    - descricao: string, opcional.
  - Resposta: HTTP 201, com os dados da publicação criada.
  - Mensagens de erro: HTTP 401, não autorizado.
  - Efeito assíncrono: emite o evento publicacao.criada ao RabbitMQ.
- Listagem de publicações — GET /publicacoes
  - Parâmetros de entrada (opcionais): categoria; status (padrão disponível); page; limit.
  - Resposta: HTTP 200, com a lista de publicações e metadados de paginação.
- Publicações do usuário autenticado — GET /publicacoes/minhas, protegido
  - Parâmetros de entrada: nenhum (identidade obtida do token).
  - Resposta: HTTP 200, com a lista de publicações do usuário.
  - Mensagens de erro: HTTP 401, não autorizado.
- Detalhe de uma publicação — GET /publicacoes/:id
  - Parâmetros de entrada: id da publicação, na URL.
  - Resposta: HTTP 200, com os dados da publicação.
  - Mensagens de erro: HTTP 404, publicação não encontrada.
- Atualização de publicação — PATCH /publicacoes/:id, protegido
  - Parâmetros de entrada (opcionais): item\_oferto; item\_desejado; categoria; descricao.
  - Resposta: HTTP 200, com os dados atualizados.
  - Mensagens de erro: HTTP 403, usuário não é o dono; HTTP 400, publicação não está disponível; HTTP 404, publicação não encontrada.
  - Efeito assíncrono: emite o evento publicacao.atualizada.
- Remoção de publicação — DELETE /publicacoes/:id, protegido
  - Parâmetros de entrada: id da publicação, na URL.
  - Resposta: HTTP 200, confirmando a remoção.

- Mensagens de erro: HTTP 403, usuário não é o dono; HTTP 404, publicação não encontrada.
    - Efeito: remoção lógica (status = removido); emite o evento publicacao.removida.
  - Interface assíncrona (consumidor RabbitMQ)
    - Eventos consumidos: match.aceito, match.encontrado, match.recusado, match.expirado, match.cancelado.
    - Efeito: atualiza o status das publicações envolvidas conforme o evento recebido.
- **Sv. Match**
- Propostas do usuário autenticado — GET /match/minhas, protegido
    - Parâmetros de entrada: nenhum (identidade obtida do token).
    - Resposta: HTTP 200, com a lista de propostas do usuário.
    - Mensagens de erro: HTTP 401, não autorizado.
  - Detalhe de uma proposta — GET /match/:id, protegido
    - Parâmetros de entrada: id da proposta, na URL.
    - Resposta: HTTP 200, com os dados da proposta.
    - Mensagens de erro: HTTP 404, proposta não encontrada; HTTP 403, usuário não participa da proposta.
  - Aceitar proposta — POST /match/:id/aceitar, protegido
    - Parâmetros de entrada: id da proposta, na URL.
    - Resposta: HTTP 200, com a proposta atualizada.
    - Mensagens de erro: HTTP 400, proposta não está mais pendente.
    - Efeito assíncrono: emite o evento match.aceito quando ambos os participantes aceitam.
  - Recusar proposta — POST /match/:id/recusar, protegido
    - Parâmetros de entrada: id da proposta, na URL.
    - Resposta: HTTP 200, com a proposta atualizada.
    - Mensagens de erro: HTTP 400, proposta não está mais pendente.
    - Efeito assíncrono: emite o evento match.recusado imediatamente.
  - Interface assíncrona (consumidor RabbitMQ)
    - Eventos consumidos: publicacao.criada, publicacao.atualizada, publicacao.removida.
    - Eventos emitidos: match.encontrado, match.aceito, match.recusado, match.expirado, match.cancelado.
- **Sv. Notificação**
- Notificações do usuário autenticado — GET /notificacoes, protegido
    - Parâmetros de entrada: nenhum (identidade obtida do token).
    - Resposta: HTTP 200, com a lista de notificações do usuário.

- Mensagens de erro: HTTP 401, não autorizado.
- Marcar notificação como lida — PATCH /notificacoes/:id/lida, protegido
  - Parâmetros de entrada: id da notificação, na URL.
  - Resposta: HTTP 200, com a notificação atualizada.
  - Mensagens de erro: HTTP 404, notificação não encontrada; HTTP 403, notificação não pertence ao solicitante.
- Interface assíncrona (consumidor RabbitMQ)
  - Eventos consumidos: match.encontrado, match.aceito, match.recusado, match.expirado, match.cancelado.
  - Efeito: cria uma notificação para cada um dos dois usuários envolvidos e a envia em tempo real via WebSocket.
- Interface WebSocket
  - Conexão: o cliente envia o token JWT no handshake; em caso de token válido, é associado ao seu canal de notificações e recebe imediatamente as notificações ainda não lidas; em caso de token ausente ou inválido, a conexão é encerrada.
  - Evento notificacao (servidor → cliente): entrega ao usuário o registro completo da notificação, sempre que uma é criada a partir de um evento de match.